

Formattatori e Time Cheatsheet

0 - Index

In tutta la gestione moderna di date, orari e formattazioni in Java, **non si usa (quasi) mai la parola chiave `new`**. Gli oggetti vengono costruiti indirettamente tramite metodi *Factory* statici.

- Nomi tipici dei metodi: `of(...)`, `now()`, `getInstance()`, `from(...)`.

Cultura Locale (`java.util.Locale`)

La cultura locale definisce le regole di formattazione (lingua e paese).

- **Costanti predefinite:** `Locale.ITALY`, `Locale.US`, `Locale.UK`

Formattatori Numerici (`java.text.NumberFormat`)

Si usano per numeri, percentuali e valute.

Creazione (Factory Methods):

- Numeri: `NumberFormat.getNumberInstance(locale)`
- Percentuali: `NumberFormat.getPercentInstance(locale)`
- Valute: `NumberFormat.getCurrencyInstance(locale)`

Utilizzo Pratico:

- **Formattare (Output):** `formatter.format(valore)`.
- **Parsing (Input):** `formatter.parse(stringa)` restituisce un oggetto `Number`. Lo estrai con `.doubleValue()`, `.intValue()`, ecc..

Pattern Personalizzati (`DecimalFormat`):

Se il default non va bene, puoi usare (qui sì!) il costruttore:

`DecimalFormat f = new DecimalFormat("¤ #,##0.¤¤; -¤ #,##0.¤¤");` f è già un formattatore

- `¤` : Simbolo valuta
- `#` : Cifra generica (può mancare)
- `0` : Cifra obbligatoria (mette gli zeri davanti)
- Il punto e la virgola nel pattern anglosassone verranno poi sostituiti in automatico dai separatori della cultura locale (es. per l'Italia la `,` diventerà `.` e viceversa).

```
1  import java.text.DecimalFormat;
2
3  // Il pattern definisce: Positivi ; Negativi
4  // ¤ = Simbolo valuta, # = cifra opzionale, 0 = cifra obbligatoria
5  Locale.setDefault(Locale.ITALY); // non dovrebbe servire
6  String pattern = "¤ #,##0.00; -¤ #,##0.00";
7  DecimalFormat valutaCustom = new DecimalFormat(pattern);
8
9  double prezzo = 1234.5;
10 double debito = -50.75;
11
12 System.out.println(valutaCustom.format(prezzo)); // Output: € 1.234,50
13 System.out.println(valutaCustom.format(debito)); // Output: -€ 50,75
14
15 prezzo = Formatters.valutaCustom.parse("€ 1.234,50").doubleValue();
```

Attention

I formattatori generano eccezioni
Per sapere quali basta fermarsi col mouse sul metodo e premere F2 o fn+F2

4. Date e Orari (`java.time`)

Tutti gli oggetti temporali sono **immutabili**. Qualsiasi modifica crea un *nuovo* oggetto, non altera quello originale!

Classe	Cosa modella	Esempio Costruzione
<code>LocalDate</code>	Data locale (giorno, mese, anno)	<code>LocalDate.of(2026, 3, 4)</code> o <code>LocalDate.now()</code>
<code>LocalTime</code>	Orario locale (ore, min, sec)	<code>LocalTime.of(12, 0)</code> o <code>LocalTime.now()</code>
<code>LocalDateTime</code>	Data + Orario	<code>LocalDateTime.of(data, orario)</code>
<code>Instant</code>	Punto fisico assoluto nel tempo	<code>Instant.now()</code>

Classe	Cosa modella	Esempio Costruzione
<code>ZonedDateTime</code>	Data + Orario + Fuso orario	<code>ZonedDateTime.of(ldt, ZoneId.of("CET"))</code>

Aritmetica e Manipolazione (Fluent Interface):

- **Aggiungere:** `.plusDays()`, `.plusMonths()`, `.plusHours()`
- **Sottrarre:** `.minusDays()`, `.minusYears()`
- **Sostituire/Impostare:** `.withYear()`, `.withMinute()`
- **Confrontare:** `.isBefore()`, `.isAfter()`, `.isEqual()`
- **Durate e Periodi:**
 - **Period:** Intervallo concettuale in anni/mesi/giorni. Creazione: `Period.between(data1, data2)` o `Period.ofMonths(2)`.
 - **Duration:** Intervallo esatto in secondi/nanosecondi. Creazione: `Duration.between(tempo1, tempo2)`.

```

1  import java.time.*;
2  import java.time.format.*;
3  import java.util.Locale;
4
5  // 1. Costruisci i pezzi base
6  LocalDate dataVolo = LocalDate.of(2026, Month.MARCH, 25);
7  LocalTime oraVolo = LocalTime.of(14, 30);
8
9  // 2. Unisci tutto con il fuso orario di partenza (es. Roma)
10 ZonedDateTime partenza = ZonedDateTime.of(dataVolo, oraVolo, ZoneId.of("Europe/Rome"));
11
12 // 3. Formattatore FULL per vedere tutti i dettagli
13 DateTimeFormatter fmtVolo = DateTimeFormatter
14     .ofLocalizedDateTime(FormatStyle.FULL, FormatStyle.LONG)
15     .withLocale(Locale.ITALY);
16
17 System.out.println(partenza.format(fmtVolo));
18 // Output: mercoledì 25 marzo 2026, 14:30:00 CET

```

Formattazione Date e Orari `java.time.format.DateTimeFormatter`

La classe principale è `DateTimeFormatter`. Anche qui si usano i Factory Methods.

Stili predefiniti (`FormatStyle`): Ci sono 4 stili: `SHORT`, `MEDIUM`, `LONG`, `FULL`.

- **Date:** Accettano tutti e 4 gli stili.
- **Orari:** Accettano SOLO `SHORT` e `MEDIUM`.

Creare e Configurare il Formatter:

```

1  // Formattatore misto: Data estesa, orario breve
2  var fmt = DateTimeFormatter.ofLocalizedDateTime(FormatStyle.LONG, FormatStyle.SHORT)
3              .withLocale(Locale.ITALY); // Specifica la lingua

```

(Nota: `.withLocale()` si applica dopo aver creato il formatter).

Pattern Personalizzati:

Se ti serve ad esempio l'anno su 4 cifre nel formato corto (cosa non supportata di default), crea un pattern:

```
DateTimeFormatter.ofPattern("dd/MM/yyyy")
```

- `y` (anno), `M` (mese come numero/testo), `d` (giorno del mese), `H` (ora 0-23), `m` (minuto), `s` (secondo).

Parsing Sicuro di Date/Orari: Esistono due approcci, ma usa sempre il **Secondo Approccio (più comodo e sicuro)** raccomandato dalle slide:

```

1  // MODO OTTIMALE: Chiedere alla classe di destinazione di fare il parsing
2  DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");
3  LocalDate data = LocalDate.parse("12/02/2026", fmt);

```

(Evita il "primo approccio" `fmt.parse("...")` perché restituisce un generico `TemporalAccessor` che va poi faticosamente convertito tramite i metodi `from(...)`).

Creare e Formattare Date e Orari Assoluti (Fusi Orari)

```

1  import java.time.*;
2  import java.time.format.*;
3  import java.util.Locale;
4
5  // 1. Costruisci i pezzi base
6  LocalDate dataVolo = LocalDate.of(2026, Month.MARCH, 25);
7  LocalTime oraVolo = LocalTime.of(14, 30);

```

```

8
9 // 2. Unisci tutto con il fuso orario di partenza (es. Roma)
10 ZonedDateTime partenza = ZonedDateTime.of(dataVolo, oraVolo, ZoneId.of("Europe/Rome"));
11
12 // 3. Formattatore FULL per vedere tutti i dettagli
13 DateTimeFormatter fmtVolo = DateTimeFormatter
14     .ofLocalizedDateTime(FormatStyle.FULL, FormatStyle.LONG)
15     .withLocale(Locale.ITALY);
16
17 System.out.println(partenza.format(fmtVolo));
18 // Output: mercoledì 25 marzo 2026, 14:30:00 CET

```

Giorno Bisestile

```

1 import java.time.*;
2
3 // La notte del cambio dell'ora legale in Europa (le 02:00 diventano le 03:00)
4 ZonedDateTime primaDelCambio = ZonedDateTime.of(
5     2026, 3, 28, 12, 0, 0, 0, ZoneId.of("Europe/Rome")
6 );
7
8 // Aggiungiamo 1 giorno logico (Period) e 24 ore fisiche (Duration)
9 ZonedDateTime domaniLogico = primaDelCambio.plus(Period.ofDays(1));
10 ZonedDateTime domaniFisico = primaDelCambio.plus(Duration.ofHours(24));
11
12 System.out.println("Domani alla stessa ora (Period): " + domaniLogico);
13 // Output: 2026-03-29T12:00+02:00[Europe/Rome] -> Sono sempre le 12:00!
14
15 System.out.println("24 ore fisiche esatte dopo (Duration): " + domaniFisico);
16 // Output: 2026-03-29T13:00+02:00[Europe/Rome] -> Sono diventate le 13:00 per via del salto!

```

Parsing "Comodo e Sicuro" di Stringhe in Date

```

1 import java.time.LocalDate;
2 import java.time.format.DateTimeFormatter;
3 import java.util.Locale;
4
5 // Stringa in input fornita dall'utente o da un file
6 String inputData = "15-ottobre-2026";
7
8 // Creiamo un pattern esatto:
9 // dd = giorno a due cifre, MMMM = mese testuale per esteso, yyyy = anno a 4 cifre
10 DateTimeFormatter parserCustom = DateTimeFormatter.ofPattern("dd-MMMM-yyyy", Locale.ITALY);
11
12 // Approccio SICURO: chiediamo a LocalDate di fare il lavoro
13 LocalDate dataConvertita = LocalDate.parse(inputData, parserCustom);
14
15 System.out.println("Data parsata con successo: " + dataConvertita);
16 // Output: 2026-10-15

```

Duration e period

```

1 import java.time.LocalDate;
2 import java.time.Period;
3
4 LocalDate oggi = LocalDate.now();
5 LocalDate compleanno = LocalDate.of(2000, 11, 20); // 20 Novembre
6
7 // Spostiamo l'anno del compleanno all'anno corrente
8 LocalDate prossimoCompleanno = compleanno.withYear(oggi.getYear());
9
10 // Se quest'anno è già passato, aggiungiamo 1 anno
11 if (prossimoCompleanno.isBefore(oggi) || prossimoCompleanno.isEqual(oggi)) {
12     prossimoCompleanno = prossimoCompleanno.plusYears(1);
13 }
14
15 Period quantoManca = Period.between(oggi, prossimoCompleanno);
16
17 System.out.println("Mancano: " + quantoManca.getMonths() + " mesi e " + quantoManca.getDays() + " giorni.");

```